

Redemption Store Integration

Engineering Analysis Summary

April 2026 | Stack: Java 21 · Spring Boot 3.2.5 · PostgreSQL 16 · React 18 · TypeScript

1. Executive Summary

The Redemption Store closes the earn-to-redeem loop on tenXengage using **XTRM** (cash payouts, 180+ countries) and **Xoxoday** (non-cash catalog). Both are fulfillment engines — tenXengage owns the authoritative ledger and the end-to-end UX. The core architectural differentiator is the **ledger-first, two-step balance model**: immutable LedgerEntry written before every wallet mutation; reserve-on-submit / debit-on-confirm prevents double-spend independent of vendor uptime.

Both documents are substantially aligned. The Prompt Engineering Doc covers entity model, all service classes, vendor adapters, webhook handlers, returns engine, and partner-facing frontend in implementation-ready prompts. **Two material gaps exist: Kafka event publishing and the admin UI layer.** These must be resolved before Sprint 1 stories are pulled.

2. Top 3 Alignment Gaps — BRD vs. Prompt Engineering Doc

Gap	BRD Coverage	Prompt Doc Coverage	Required Action
Kafka Event Publishing (15 event types)	Fully defined — Section 13. Topics: redemption_*, wallet_*, vendor_webhook_*.	NOT COVERED. No producer class, no topic names, no schema format in any prompt.	Add RedemptionEventPublisher prompt before Sprint 1. Lock topic names and schema (JSON) with platform team.
Admin UI Layer — Client Admin + Platform Admin	Fully defined — Section 24. 3 Client Admin views, 2 Platform Admin views.	NOT COVERED. Only partner-facing pages prompted (Sections 5.2–5.5).	Add 3 Client Admin prompts (Approval Queue, Catalog Config, Tenant Reporting) + Platform Admin catalog manager to Prompt Doc.
Notification Service Integration (11 event triggers)	Fully defined — Section 19. All events, recipients, and category mapped.	NOT COVERED. No notification publish call in any service or webhook handler prompt.	Add explicit notification publish calls to webhook handlers and service methods. Confirm interface contract with notification team.

3. Architecture Decisions to Lock Before Sprint 1

Decision	What Breaks If Deferred
Dead-letter strategy for unprocessable webhooks	DB table recommended (redemption_webhook_dlq). BRD mandates DLQ; Prompt Doc only says "log." Wallets can be left in suspended state with no recovery path.
Kafka event publishing spec	Downstream notification, analytics, and PAS pipeline have no integration point from day 1.
Pessimistic vs. optimistic locking on reserve()	Ambiguity in Prompt Doc 4.1. Pessimistic write lock required on reserve(). Wrong choice enables double-spend under concurrent submissions.
Vendor idempotency key confirmation (XTRM + Xoxoday)	@Retryable re-submits same UUID as externalId. Must confirm vendors honor it for deduplication before adapter build. Duplicate cash payouts are the failure mode.
EXPIRY_DEBIT removal from V002 migration	Phase 2 concern shipped in Sprint 1 schema. Hard to remove cleanly post-migration without risk.
Reward earning → WalletService.credit() integration path	If existing incentive earning events do not call credit(), wallets never receive balances and the store is inoperable at launch.
Platform Admin backend catalog endpoints	No CRUD API for global catalog management. Platform team must use direct DB writes post-launch.

4. Partner Readiness Score Design

Finding: The Partner Readiness Score (PRS) and CertificationStatus enum belong to the **Enablement** quadrant, not the Redemption Store. Neither document defines a PRS for this feature.

What this feature contributes to scoring is the **PAS Commercial Intent signal (18% weight, Phase 2)**: redemption_completed and wallet_balance_updated events route into the ActivationSignal pipeline. This is correctly deferred to Phase 2 in both documents. Sprint 1 prerequisite: Kafka event publishing must be implemented so the signal exists when Phase 2 integrates it. **No CertificationStatus enum is defined or required here.**

5. Sprint Readiness Rating

Rating: READY WITH CONDITIONS

Open conditions (all must close before Sprint 1 stories move to In Progress):

- **Kafka:** RedemptionEventPublisher spec added to Prompt Doc; topic names and schema agreed with platform team.
- **Dead-letter:** Webhook DLQ strategy decided and implemented in both webhook handler services.
- **Admin UI:** Client Admin and Platform Admin frontend prompts added to Prompt Doc and scoped into sprint.
- **Schema:** EXPIRY_DEBIT removed from V002 Flyway migration (Phase 2 concern).
- **Sandbox:** XTRM and Xoxoday sandbox credentials provisioned and stored in secrets manager.
- **Earning path:** Reward earning → WalletService.credit() integration point confirmed with Incentives/Engagement PODs.
- **Platform Admin API:** Backend endpoints for global catalog management specified and added to Prompt Doc.

6. Top 5 Risks

Risk	Prob	Impact	Mitigation
Kafka gap blocks all downstream integrations	H	H	Add RedemptionEventPublisher in next 8h. Lock topic names with platform infra today.
Concurrent reserve() race condition → double-spend	M	H	Confirm @Lock(PESSIMISTIC_WRITE) on wallet SELECT inside reserve(). Add concurrency integration test.
Webhook DLQ undefined → irrecoverable wallet state	M	H	Decide DB vs. Kafka DLQ today. Implement before webhook handlers are code-reviewed.
XTRM/Xoxoday sandbox not provisioned → adapter blocked	M	H	Escalate to PM immediately. Adapter dev cannot start without sandbox credentials.
Admin UI unscoped → Client Admins cannot approve at launch	M	H	Add admin frontend prompts to Prompt Doc before Sprint 1 planning. Scope into sprint backlog.

7. Recommended Next Actions (48–72 hrs)

When	Action	Owner
0–4 h	Remove EXPIRY_DEBIT from V002 migration draft in Prompt Doc Section 3.2.	Lead Backend Eng
0–8 h	Decide webhook dead-letter strategy (DB table) and add to Prompt Doc as Section 4.5 addendum.	Tech Lead
0–8 h	Define all 15 Kafka topic names and JSON schema. Add RedemptionEventPublisher prompt to Prompt Doc.	Tech Lead

When	Action	Owner
0–8 h	Contact XTRM and Xoxoday vendor contacts for sandbox API keys and webhook signing secrets.	PM + Backend Lead
8–24 h	Confirm reward earning → WalletService.credit() integration path with Incentives/Engagement PODs.	Tech Lead
8–24 h	Add Client Admin and Platform Admin frontend prompts to Prompt Doc Section 5.	Lead Frontend Eng
24–48 h	Execute Prompt Doc steps 1–4 (data model). Review all 6 entities + migrations against checklist.	Full Backend Team

Reference document: TenXEngage_Engineering_Analysis_Full.docx | This summary will be compared against the Prompt Engineering Document at sprint end.